

Transformer Models Documentation

GitHub Repo: <https://github.com/abiddileepjan7-prog/deberta-text-classification>

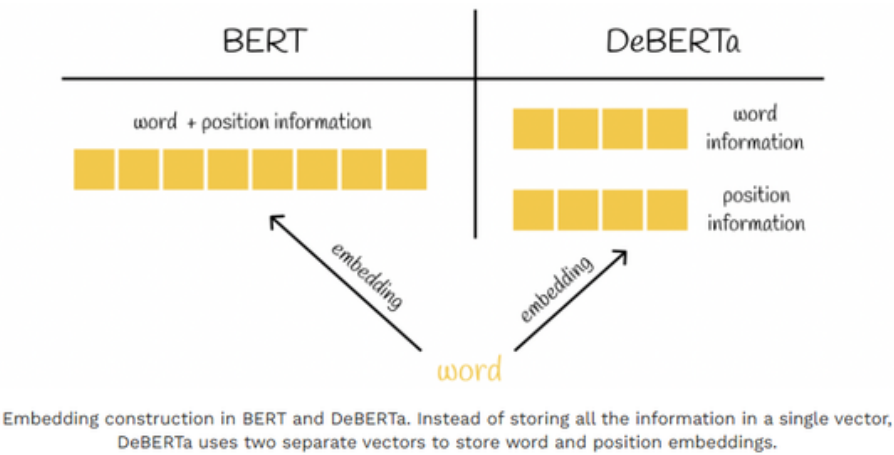
Architecture	Model	Task
Encoder-only	DeBERTa-v3 Small	Sentiment Classification
Decoder-only	SmolLM2-360M-Instruct	Text Generation
Encoder-Decoder	LED (Longformer Encoder Decoder)	Text Summarization

1. Encoder-only Model: DeBERTa-v3 Small

Model Name	mrm8488/deberta-v3-small-finetuned-sst2
Model Family	DeBERTa (Decoding-enhanced BERT with Disentangled Attention)
Developed By	Microsoft Research
Imported From	Hugging Face Model Hub
Model Identifier	model="mrm8488/deberta-v3-small-finetuned-sst2"
Architecture	Encoder-only Transformer
Base Model	microsoft/deberta-v3-small (fine-tuned on SST-2 sentiment analysis dataset)

DeBERTa stands for Decoding-enhanced BERT with Disentangled Attention. It is an encoder-only transformer developed by Microsoft to improve upon BERT's understanding of language.

Unlike traditional BERT, DeBERTa separates the representation of a word from its position in the sentence, allowing it to capture contextual relationships more effectively.



$$A_{ij} = \left\{ \underbrace{H_i}_{\text{content } i} \underbrace{P_{i|j}}_{\text{relative position of } i \text{ with } j} \right\} \times \left\{ \underbrace{H_j}_{\text{content } j} \underbrace{P_{j|i}}_{\text{relative position of } j \text{ with } i} \right\}^T =$$

$$= \underbrace{H_i H_j^T}_{\text{content-to-content matrix}} + \underbrace{H_i P_{j|i}^T}_{\text{content-to-position matrix}} + \underbrace{P_{i|j} H_j^T}_{\text{position-to-content matrix}} + \underbrace{P_{i|j} P_{j|i}^T}_{\text{position-to-position matrix}}$$

Computation of cross-attention score between two embedding vectors.

These are the difference from the normal :

- Separate Word Embedding and Position Embedding
- Disentangled Self-Attention
- Relative Position Encoding



Input Sentence → Tokenization → Token IDs → Content Embeddings + Relative Position Embeddings → Disentangled Multi-Head Self-Attention → Add & Layer Normalization → Feed Forward Network (FFN) → Add & Layer Normalization → Repeat for 6 Encoder Layers → Final Contextual Embeddings → Task-Specific Head → Prediction

- **Tokenization:** Splits the input text into smaller units (tokens) and converts them into numerical IDs that the model can process.
- **Content & Relative Position Embeddings:** Generates content embeddings for each token and relative position embeddings to capture the distance between tokens without mixing the two representations.
- **Disentangled Multi-Head Self-Attention:** Computes attention by separately considering token content and relative positions, allowing the model to better understand semantic and positional relationships.
- **Add & Layer Normalization:** Uses residual (skip) connections to preserve information and layer normalization to stabilize training and improve convergence. **:Feed Forward Network (FFN)**
- **Applies a fully connected neural network to each token independently,** learning more complex and meaningful feature representations.
- **Final Contextual Embeddings:** Produces context-aware representations where each token's embedding reflects both its own meaning and its relationship with surrounding tokens.
- **Task-Specific Head:** Maps the contextual embeddings to the target NLP task, such as text classification, question answering, or named entity recognition.
- **Prediction:** Produces the final output, such as a class label, extracted answer, or token classification result.

Why This Model?

Lightweight

High accuracy

Optimized for understanding text

Excellent for classification tasks

Common Applications

Sentiment Analysis

Spam Detection

Topic Classification

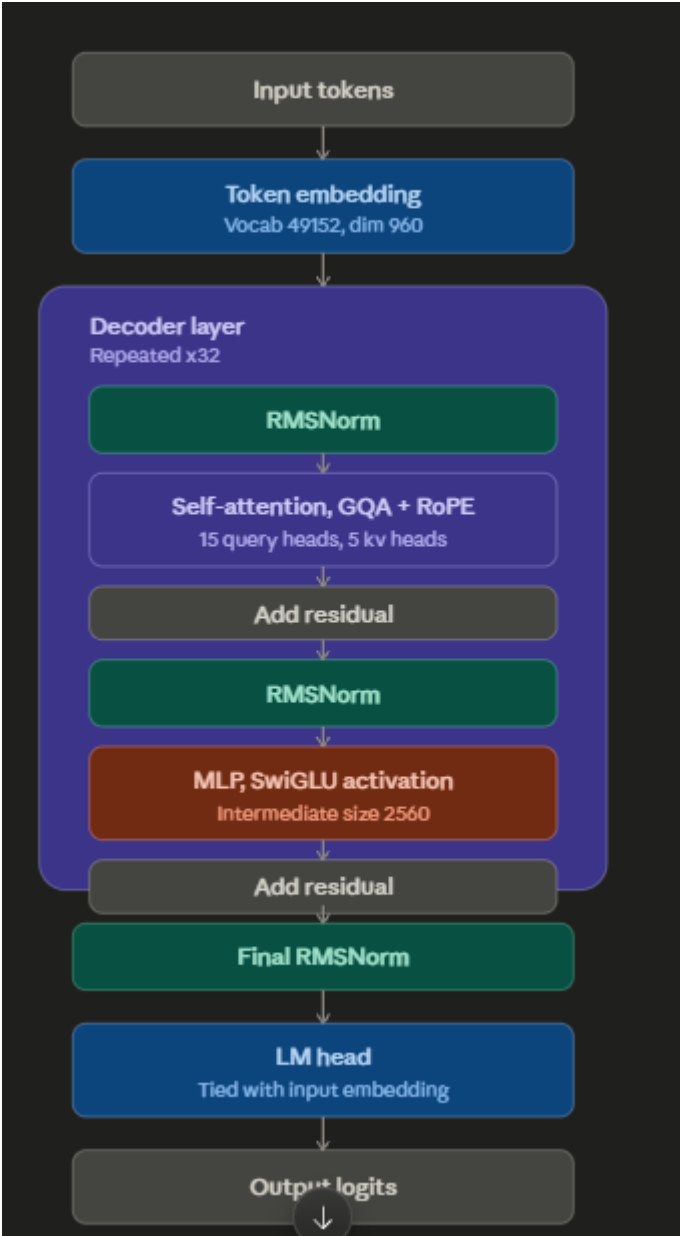
Intent Detection

Review Classification

2. Decoder-only Model: SmolLM2-360M-Instruct

Model	Name	HuggingFaceTB/SmolLM2-360M-Instruct
Model	Family	SmolLM2
Developed	By	Hugging Face
Imported	From	Hugging Face Model Hub
Model	Identifier	model="HuggingFaceTB/SmolLM2-360M-Instruct"
Architecture		Decoder-only Transformer
Parameters		Approximately 360 Million Parameters

SmolLM2 is a compact decoder-only language model developed by Hugging Face. It predicts text one token at a time using only previous tokens, making it ideal for text generation. Uses Masked Self Attention



- Standard Self-Attention --Masked (Causal) Self-Attention – Prevents seeing future tokens during generation
- Learned Positional Encoding – Rotary Positional Embeddings (RoPE) --Better handling of long-context dependencies
- Multi-Head Attention – Grouped Query Attention (GQA) – Reduces KV cache memory and speeds up inference
- Standard FFN – SwiGLU Feed Forward Network – Better feature learning than standard FFN
- Layer Normalization – RMSNorm --Faster and more stable training than LayerNorm
- Standard Attention Implementation – Flash Attention (when supported) --Faster and more memory-efficient attention computation

RMS Norms:

Root mean square normalization

- type of positional encoding. Rope uses query value vectors . like arrows to vectors
- insted of normal normailization it uses only root mean square.
- it is faster for large models
- memory efficient
- better understanding

Position 1

→

Position 2

↗

Position 3

↑

Position 4

↖

AUTO REGRESSIVE MODEL

this model predicts one token at a time with the help of only tokens that are generated.

does not see the future word

Imagine you want to complete the sentence:

The cat sat on the _____

The model predicts

mat

After generating "mat", the input becomes

The cat sat on the mat

Now it predicts the next word.

The cat sat on the mat and

Then

The cat sat on the mat and slept.

The model repeats this until it reaches an end-of-sequence token

3. Encoder-Decoder Model: LED (Longformer Encoder Decoder)

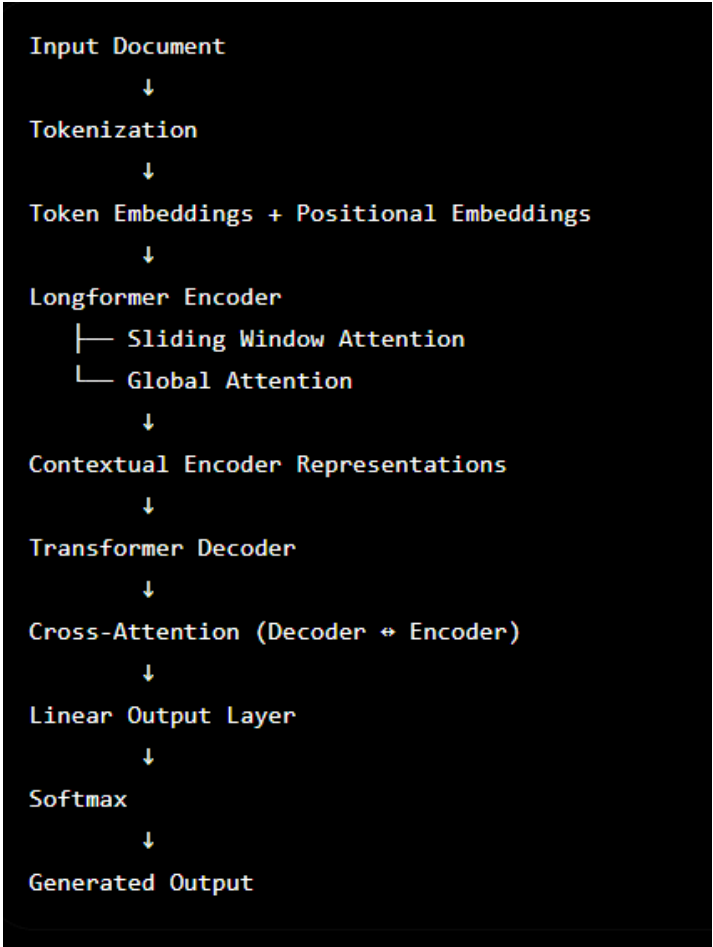
Model Name		allenai/led-base-16384
Model	Family	LED (Longformer Encoder Decoder)
Developed	By	Allen Institute for AI (AI2)
Imported From		Hugging Face Model Hub
Model	Identifier	model="allenai/led-base-16384"
Architecture		Encoder-Decoder Transformer
Maximum Input Length		Up to 16,384 tokens

LED (Longformer Encoder Decoder) is an encoder-decoder model developed by the Allen Institute for AI. It extends the Longformer architecture to efficiently process very long documents, making it well suited for long-document summarization.

Instead of standard self-attention, LED uses:

- Local Attention: each token attends only to nearby tokens, reducing computation.
- Global Attention: selected important tokens (such as the first token or special markers) attend to all tokens and receive information from all tokens.

This combination allows LED to handle much longer inputs than standard transformers.



Tokenization: Splits the long document into tokens and converts them into token IDs.

Token Embeddings + Positional Embeddings: Converts token IDs into dense vectors and adds positional information.

Sliding Window Attention : Each token attends only to nearby tokens within a fixed window, greatly reducing computation.

Global Attention : Selected important tokens (e.g., question tokens, special tokens) can attend to every token in the document and vice versa.

Longformer Encoder : Builds contextual representations using both local and global attention mechanisms.

Transformer Decoder: Generates the output sequence one token at a time using masked self-attention and encoder-decoder cross-attention.

Cross-Attention :Allows the decoder to attend to the encoder's contextual representations while generating the output.

Advantages of LED

- Efficiently processes very long documents.
- Significantly lower memory consumption than full self-attention.
- Faster than a standard Transformer on long sequences.
- Retains important global information using global attention.
- Ideal for document summarization and long-context NLP tasks.
- Maintains the encoder-decoder architecture, making it suitable for text generation tasks.

Transformer Attention Mechanisms

1. Transformer Architecture

A Transformer is a neural network design that processes sequences (like sentences) by looking at all the words at once, instead of one at a time like older models (RNNs). It uses a mechanism called **attention** to decide which words in a sequence matter most to each other.

Main Building Blocks

- **Embeddings** — turn words into numbers (vectors) the model can process
- **Positional Encoding** — adds information about word order, since attention has no built-in sense of order
- **Multi-Head Attention** — lets the model focus on different parts of a sentence at the same time
- **Feed-Forward Layers** — small neural networks that further process each word's information
- **Residual Connections & Normalization** — help the model train smoothly and avoid losing information

Two Main Parts

Part	Role
Encoder	Reads and understands the input (e.g., a sentence to translate)
Decoder	Generates output step by step (e.g., the translated sentence)

2. Multi-Head Attention (MHA)

Attention works using three vectors for each word: **Query (Q)**, **Key (K)**, and **Value (V)**. The model compares the Query of one word to the Keys of all words to figure out how relevant they are, then blends their Values based on that relevance.

'Multi-head' means the model does this attention process several times in parallel, each time with a different learned perspective (a 'head'). One head might focus on grammar, another on meaning, another on word order — then all the results are combined.

Key Points

- Each head has its own Query, Key, and Value projections
- Multiple heads let the model capture different types of relationships at once
- Very powerful, but requires storing full Key/Value sets for every head — this is expensive in memory and speed

Formula (simplified): $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d}) V$

3. Multi-Query Attention (MQA)

MQA is a faster, lighter version of Multi-Head Attention. Instead of giving every head its own Key and Value, **all heads share a single Key and Value**, while each head still keeps its own Query.

Key Points

- Much less memory needed, since Key/Value are stored only once instead of per head
- Faster inference — especially helpful for generating long text
- Slight drop in model quality/accuracy compared to full Multi-Head Attention

4. Grouped Query Attention (GQA)

GQA is a middle ground between MHA and MQA. Instead of every head having its own Key/Value (MHA) or all heads sharing one Key/Value (MQA), heads are split into **small groups**, and each group shares one Key/Value.

Key Points

- Balances speed and quality between MHA and MQA
- Used in many modern large language models (e.g., LLaMA 2/3, Mistral)
- Gives most of MQA's speed benefits while keeping accuracy close to MHA

5. Flash Attention

FlashAttention is not a new type of attention calculation — it's a **faster way to compute the same attention math** on GPUs. It changes how the computation is done in hardware memory, not what is computed.

Normal attention creates a huge intermediate table of scores that has to be written to and read from slow GPU memory. Flash Attention breaks the computation into small chunks and processes them in fast on-chip memory, avoiding that costly back-and-forth.

Key Points

- Produces mathematically identical results to standard attention
- Much faster and uses significantly less memory
- Enables training and running models on much longer sequences
- Can be combined with MHA, MQA, or GQA — it's a computation trick, not a Q/K/V design change

Attention Components Guide

1. Local Self-Attention (Sliding Window Attention)

In standard self-attention, every word looks at every other word in the sequence. This works well for short text, but becomes very slow and memory-hungry for long documents, since the cost grows quadratically with sequence length.

Local self-attention solves this by having each word only look at a small window of nearby words — for example, the 256 words before and after it — instead of the entire document.

Key Points

- Each token attends only to a fixed-size neighborhood (the 'window') around it
- Cost grows linearly with sequence length instead of quadratically
- Captures short-range relationships well (grammar, nearby phrases)
- Misses long-range relationships on its own (e.g., connecting the first and last paragraph)
- Used in models like Longformer and LED to handle very long documents efficiently

Local attention trades some long-range understanding for a big gain in speed and memory efficiency, which is essential when processing documents with thousands of tokens.

2. Global Self-Attention

Since local attention alone can't connect distant parts of a long document, models like Longformer add **global attention** on top of it. A small number of important tokens are marked as 'global' — these tokens attend to every other token in the sequence, and every other token attends back to them.

Think of it like a class discussion with a small flashlight rule for most students, but the teacher (the global token) can see and be seen by everyone in the room at once, connecting ideas from the front of the class to the back.

Which Tokens Get Global Attention?

- Special tokens like the classification token (commonly written as [CLS] or <s>)
- Task-specific tokens, such as question tokens in question-answering tasks
- A small, chosen set of positions — not the whole sequence, to keep costs low

Key Points

- Global tokens act as information 'hubs' that summarize and spread context across the document
- Combined with local attention, the model gets both short-range detail and long-range context
- Only a few tokens use global attention, so the overall cost stays close to linear

3. Cross-Attention

Local and global attention are both forms of **self-attention** — a sequence attending to itself.

Cross-attention is different: it lets one sequence attend to a completely different sequence. This is the mechanism that connects the encoder and the decoder in encoder-decoder models.

In cross-attention, the **Query** comes from the decoder (what is currently being generated), while the **Key** and **Value** come from the encoder's output (the full understanding of the input document). This lets the decoder ask, 'given what I'm writing now, which parts of the input document are most relevant?'

Like a translator who has fully read a foreign article (the encoder's output) and, while writing each sentence of the translation (the decoder), keeps glancing back at the parts of the article that matter most for the sentence being written right now.

Key Points

- Query vectors come from the decoder; Key and Value vectors come from the encoder
- Lets the decoder focus on the most relevant parts of the input for each generated token
- Essential for tasks like translation and summarization, where output must stay grounded in input
- Used in every decoder layer of models like LED, T5, and the original Transformer

Formula (same attention math as self-attention, but Q comes from a different sequence than K and V): $\text{Attention}(Q_{\text{decoder}}, K_{\text{encoder}}, V_{\text{encoder}}) = \text{softmax}(QK^T / \sqrt{d}) V$

Comparison Summary

Component	Attends To	Sequence Relationship	Main Purpose
Local self-attention	Nearby tokens in a fixed window	Same sequence	Efficient short-range context
Global self-attention	Every token (for select tokens)	Same sequence	Long-range context via hub tokens
Cross-attention	All tokens in the encoder output	Two different sequences	Ground decoder output in the input

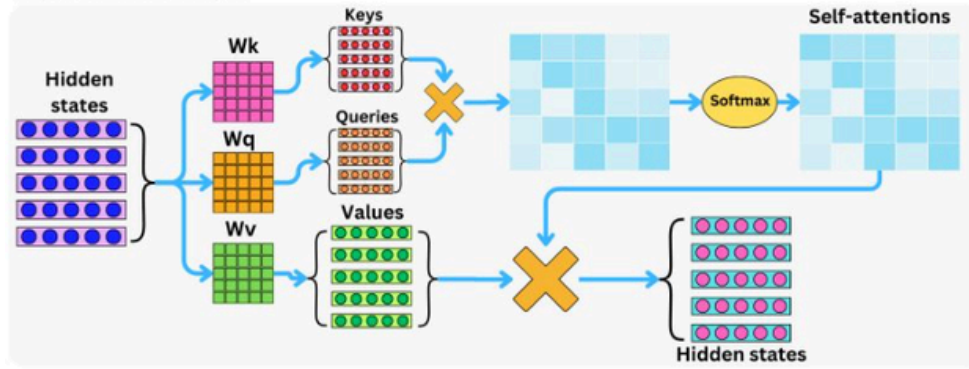
In short: local and global attention work together inside the encoder of long-document models like LED, balancing speed and context coverage. Cross-attention is a separate mechanism that lets the decoder pull relevant information from the encoder while generating output, one token at a time.

In cross attention the decoder communicates with encoder before generating next word

- Query comes from the decoder The encoder gives the key and value
- same equation as self attention Q,K,V

Self-Attention VS Cross-Attention TheAiEdge.io

The Self-Attentions



The Cross-Attentions

